

Sumário

Exercícios de MongoDB — Explicados	1
Preparação do ambiente	1
Exercício 1 — find() com condição (\$gt)	2
Exercício 2 — Projeção	2
Exercício 3 — Agregação com \$sum	2
Exercício 4 — Agrupamento por campo	2
Exercício 5 — \$avg (média)	3
Exercício 6 — \$sort (ordenação)	3
Exercício 7 — updateOne	3
Exercício 8 — updateMany	3
Exercício 9 — deleteMany	3
Exercício 10 — Busca por texto simples com \$regex	4
Exercício 11 — Expressão regular com âncora	4
Exercício 12 — Intervalo de datas	4
Exercício 13 — \$lookup (junção entre coleções)	4
Exercício 14 — Criar índice	5
Exercício 15 — Agrupamento por múltiplos campos	5
Exercício 16 — Paginação (\$skip + \$limit)	5
Exercício 17 — Adicionar campo calculado (\$set com \$multiply)	6
Exercício 18 — Adicionar array aninhado	6
Exercício 19 — \$in e \$nin	6
Exercício 20 — \$and e \$or	6
Exercício 21 — \$exists	7
Exercício 22 — distinct	7
Exercício 23 — \$min e \$max	7
Exercício 24 — \$project com campos calculados	7
Exercício 25 — \$count	8
Exercício 26 — \$unwind	8
Exercício 27 — Aggregation de várias etapas	8
Exercício 28 — \$addToSet e contagem de únicos	9
Exercício 29 — Índice composto	9
Exercício 30 — insertOne e insertMany	9
Bônus — Boas práticas ao resolver	9

Exercícios de MongoDB — Explicados

Guia prático de exercícios para alunos. Para cada tópico: 1. **O que a função faz** — explicação teórica. 2. **Exemplo** — código pronto com resultado esperado. 3. **Exercício** — desafio para o aluno resolver.

Preparação do ambiente

Antes de iniciar, importe as duas coleções para o banco loja:

```
mongoimport --db loja --collection pedidos --file dados_pedidos.json --jsonArray
mongoimport --db loja --collection clientes --file clientes.json --jsonArray
```

Depois, no mongosh:

```
use loja
db.pedidos.countDocuments() // deve retornar 35
db.clientes.countDocuments() // deve retornar 30
```

A coleção pedidos contém os campos: ORDERNUMBER, QUANTITYORDERED, PRICEEACH, SALES, STATUS, ORDERDATE, PRODUCTLINE, PRODUCTCODE, CUSTOMER_ID, CONTACTLASTNAME, CITY, COUNTRY.

Exercício 1 — find() com condição (\$gt)

O que faz: find(filtro) procura documentos que satisfazem o filtro. O operador \$gt (greater than) devolve documentos em que o campo é maior que o valor dado. Existem também \$lt, \$gte, \$lte, \$ne e \$eq.

Exemplo — pedidos com SALES acima de 5000:

```
db.pedidos.find({ SALES: { $gt: 5000 } })
```

Retorna todos os documentos cujo campo SALES seja maior que 5000.

Exercício para o aluno: Recupere todos os pedidos onde QUANTITYORDERED seja maior que 50.

Exercício 2 — Projeção

O que faz: O segundo parâmetro do find() define a **projeção** — quais campos devolver. 1 inclui, 0 exclui. O campo _id vem por padrão; para o omitir passe _id: 0.

Exemplo — mostrar só o número e o país:

```
db.pedidos.find({}, { ORDERNUMBER: 1, COUNTRY: 1, _id: 0 })
```

Exercício para o aluno: Retorne apenas os campos ORDERNUMBER, SALES e STATUS para todos os documentos.

Exercício 3 — Agregação com \$sum

O que faz: O estágio \$group agrupa documentos. Quando _id: null, agrupa todos num único resultado. O acumulador \$sum soma valores dos documentos agrupados.

Exemplo — total de unidades vendidas:

```
db.pedidos.aggregate([
  { $group: { _id: null, totalUnidades: { $sum: "$QUANTITYORDERED" } } }
])
```

Exercício para o aluno: Calcule o total de SALES para todos os pedidos feitos em 2003. (Dica: use \$match com um intervalo de datas antes do \$group.)

Exercício 4 — Agrupamento por campo

O que faz: Passando o nome de um campo em _id (com \$), o \$group cria um grupo por cada valor único desse campo.

Exemplo — número de pedidos por status:

```
db.pedidos.aggregate([
  { $group: { _id: "$STATUS", total: { $sum: 1 } } }
])
```

Exercício para o aluno: Agrupe os pedidos por COUNTRY e obtenha o total de SALES para cada país.

Exercício 5 — \$avg (média)

O que faz: \$avg calcula a média aritmética de um campo numérico dentro do grupo.

Exemplo — preço médio por linha de produto:

```
db.pedidos.aggregate([
  { $group: { _id: "$PRODUCTLINE", precoMedio: { $avg: "$PRICEEACH" } } }
])
```

Exercício para o aluno: Encontre a média de SALES para todos os pedidos.

Exercício 6 — \$sort (ordenação)

O que faz: \$sort (no aggregate) ou .sort() (no find) ordena os resultados. 1 = crescente, -1 = decrescente.

Exemplo — 5 maiores pedidos:

```
db.pedidos.find().sort({ SALES: -1 }).limit(5)
```

Exercício para o aluno: Recupere todos os pedidos ordenados por SALES em ordem decrescente.

Exercício 7 — updateOne

O que faz: updateOne(filtro, atualização) altera apenas o **primeiro** documento que corresponde ao filtro. O operador \$set altera valores de campos.

Exemplo:

```
db.pedidos.updateOne(
  { ORDERNUMBER: 10100 },
  { $set: { STATUS: "Delivered" } }
)
```

Exercício para o aluno: Atualize o STATUS do pedido ORDERNUMBER = 10107 para "Delivered".

Exercício 8 — updateMany

O que faz: Igual ao updateOne, mas altera **todos** os documentos que correspondem ao filtro.

Exemplo — colocar todos os pedidos cancelados em revisão:

```
db.pedidos.updateMany(
  { STATUS: "Cancelled" },
  { $set: { STATUS: "Review" } }
)
```

Exercício para o aluno: Atualize o STATUS para "Processing" em todos os pedidos onde QUANTITYORDERED < 30.

Exercício 9 — deleteMany

O que faz: Remove todos os documentos que correspondem ao filtro. **Atenção:** operação irreversível.

Exemplo:

```
db.pedidos.deleteMany({ STATUS: "Cancelled" })
```

Exercício para o aluno: Elimine todos os pedidos onde COUNTRY seja "USA". Antes de executar, faça um `countDocuments` para saber quantos serão apagados.

Exercício 10 — Busca por texto simples com \$regex

O que faz: `$regex` permite pesquisar por padrão numa string. Sem âncora, funciona como um “contains”.

Exemplo — apelidos que contenham “son”:

```
db.pedidos.find({ CONTACTLASTNAME: { $regex: "son", $options: "i" } })
```

`$options: "i"` torna a busca case-insensitive.

Exercício para o aluno: Encontre todos os pedidos com um `CONTACTLASTNAME` que contenha "Young".

Exercício 11 — Expressão regular com âncora

O que faz: `^` na regex indica **início da string**. Assim conseguimos filtrar valores que começam por certa letra.

Exemplo — cidades que começam por “M”:

```
db.pedidos.find({ CITY: { $regex: "^M" } })
```

Exercício para o aluno: Encontre todos os pedidos em que a `CITY` comece pela letra "P" usando uma expressão regular.

Exercício 12 — Intervalo de datas

O que faz: Combinamos `$gte` e `$lte` no mesmo campo para obter um intervalo. Datas podem ser guardadas como string ISO ou como `ISODate()`.

Exemplo — pedidos do 1º semestre de 2004:

```
db.pedidos.find({
  ORDERDATE: { $gte: "2004-01-01", $lte: "2004-06-30" }
})
```

Exercício para o aluno: Recupere todos os pedidos feitos entre 01/07/2003 e 31/12/2003.

Exercício 13 — \$lookup (junção entre coleções)

O que faz: `$lookup` faz o equivalente a um LEFT JOIN entre duas coleções. Para cada documento da coleção de entrada, procura documentos correspondentes em outra coleção.

Exemplo — juntar cada pedido com os dados do cliente:

```
db.pedidos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "CUSTOMER_ID",
      foreignField: "customer_id",
```

```

    as: "cliente"
  }
},
{ $limit: 3 }
])

```

Cada pedido passa a incluir um array cliente com o(s) documento(s) correspondente(s).

Exercício para o aluno: Use \$lookup para juntar pedidos com clientes e devolver ORDERNUMBER, SALES e o customer_name correspondente (dica: use \$unwind + \$project depois do \$lookup).

Exercício 14 — Criar índice

O que faz: createIndex cria um índice num campo, acelerando pesquisas e ordenações por esse campo à custa de mais escrita e espaço em disco.

Exemplo — índice único no número do pedido:

```

db.pedidos.createIndex({ ORDERNUMBER: 1 }, { unique: true })
db.pedidos.getIndexes()

```

Exercício para o aluno: Crie um índice no campo ORDERNUMBER para acelerar as pesquisas por número de pedido e depois liste os índices existentes.

Exercício 15 — Agrupamento por múltiplos campos

O que faz: _id em \$group pode ser um objeto com vários campos. Cria um grupo por cada combinação única de valores.

Exemplo — total por linha de produto e status:

```

db.pedidos.aggregate([
  {
    $group: {
      _id: { linha: "$PRODUCTLINE", status: "$STATUS" },
      total: { $sum: "$SALES" }
    }
  }
])

```

Exercício para o aluno: Agrupe os pedidos por COUNTRY e STATUS, calculando o total de SALES para cada combinação.

Exercício 16 — Paginação (\$skip + \$limit)

O que faz: \$skip(n) ignora os primeiros n documentos; \$limit(n) mantém no máximo n. Combinados com \$sort, permitem paginar.

Exemplo — segunda página de 5 pedidos, ordenados por SALES desc:

```

db.pedidos.find().sort({ SALES: -1 }).skip(5).limit(5)

```

Exercício para o aluno: Recupere os primeiros 10 pedidos com SALES > 3000, ordenados por SALES em ordem decrescente.

Exercício 17 — Adicionar campo calculado (\$set com \$multiply)

O que faz: \$set no aggregate/update permite criar um novo campo com base numa expressão. \$multiply multiplica dois ou mais valores.

Exemplo — nova coluna com custo total do item:

```
db.pedidos.updateMany(
  {},
  [ { $set: { TotalCost: { $multiply: [ "$QUANTITYORDERED", "$PRICEEACH" ] } } } ]
)
```

(A sintaxe updateMany com pipeline [...] permite usar expressões.)

Exercício para o aluno: Adicione o campo TotalCost a cada documento, com o produto de QUANTITYORDERED e PRICEEACH. Depois faça um find() para confirmar.

Exercício 18 — Adicionar array aninhado

O que faz: Um documento pode ter arrays de subdocumentos. \$set pode inserir esses arrays diretamente.

Exemplo — adicionar detalhes de produto a um pedido:

```
db.pedidos.updateOne(
  { ORDERNUMBER: 10100 },
  { $set: {
    productDetails: [
      { product_name: "1969 Harley Davidson", product_code: "S10_1678", product_description: "Mo
    ]
  } }
)
```

Exercício para o aluno: Adicione um array productDetails a **todos** os pedidos com PRODUCTCODE = "S18_1342" contendo product_name, product_code e product_description para a linha "Trucks".

Exercício 19 — \$in e \$nin

O que faz: \$in verifica se o campo está numa lista de valores; \$nin é o oposto.

Exemplo — pedidos de Itália, França ou Portugal:

```
db.pedidos.find({ COUNTRY: { $in: ["Italy", "France", "Portugal"] } })
```

Exercício para o aluno: Devolva os pedidos com STATUS que **não seja** "Shipped" nem "Cancelled" (use \$nin).

Exercício 20 — \$and e \$or

O que faz: Combinam múltiplas condições. Por padrão, várias chaves num filtro são um AND lógico; \$or requer a sintaxe explícita.

Exemplo — pedidos grandes ou de um cliente VIP:

```
db.pedidos.find({
  $or: [
    { SALES: { $gt: 8000 } },
    { CUSTOMER_ID: 4 }
  ]
})
```

```
]
})
```

Exercício para o aluno: Encontre pedidos com QUANTITYORDERED entre 20 e 40 e com COUNTRY diferente de "USA".

Exercício 21 — \$exists

O que faz: Verifica se um campo existe (ou não) no documento — útil quando o schema é flexível.

Exemplo:

```
db.pedidos.find({ TotalCost: { $exists: true } })
```

Exercício para o aluno: Após o exercício 17, execute uma query que devolva apenas documentos que **já tenham** o campo TotalCost.

Exercício 22 — distinct

O que faz: Devolve os valores únicos de um campo.

Exemplo — lista de países presentes:

```
db.pedidos.distinct("COUNTRY")
```

Exercício para o aluno: Liste todas as linhas de produto (PRODUCTLINE) únicas presentes na coleção.

Exercício 23 — \$min e \$max

O que faz: Acumuladores que devolvem o menor e o maior valor de um campo dentro do grupo.

Exemplo — menor e maior venda por país:

```
db.pedidos.aggregate([
  { $group: {
    _id: "$COUNTRY",
    menorVenda: { $min: "$SALES" },
    maiorVenda: { $max: "$SALES" }
  } }
])
```

Exercício para o aluno: Para cada PRODUCTLINE, obtenha o preço unitário mínimo e o máximo (PRICEEACH).

Exercício 24 — \$project com campos calculados

O que faz: O estágio \$project reorganiza a saída, incluindo, excluindo ou criando novos campos.

Exemplo — mostrar apenas número e receita em milhares:

```
db.pedidos.aggregate([
  { $project: {
    _id: 0,
    ORDERNUMBER: 1,
    salesK: { $divide: ["$SALES", 1000] }
  } }
])
```

```
} }  
])
```

Exercício para o aluno: Use \$project para devolver ORDERNUMBER, COUNTRY e um novo campo desconto—Sugerido igual a 10% de SALES.

Exercício 25 — \$count

O que faz: Estágio que devolve o número de documentos que chegaram até ele.

Exemplo:

```
db.pedidos.aggregate([  
  { $match: { STATUS: "Shipped" } },  
  { $count: "pedidosEnviados" }  
])
```

Exercício para o aluno: Conte quantos pedidos têm SALES superior à média global de SALES. (Dica: use \$group para calcular a média num pipeline separado, ou faça em duas queries.)

Exercício 26 — \$unwind

O que faz: Descompacta arrays. Para cada elemento do array, cria um documento separado. Útil após um \$lookup.

Exemplo:

```
db.pedidos.aggregate([  
  { $lookup: { from: "clientes", localField: "CUSTOMER_ID", foreignField: "customer_id", as: "cli" } },  
  { $unwind: "$cli" },  
  { $project: { ORDERNUMBER: 1, "cli.customer_name": 1, _id: 0 } },  
  { $limit: 5 }  
])
```

Exercício para o aluno: Depois do exercício 18, use \$unwind para “achatar” o array productDetails e listar ORDERNUMBER e productDetails.product_name para cada linha.

Exercício 27 — Aggregation de várias etapas

O que faz: Pipelines de agregação encadeiam vários estágios (\$match -> \$group -> \$sort -> \$limit ...). Cada estágio recebe a saída do anterior.

Exemplo — top 3 países com mais receita, só pedidos “Shipped”:

```
db.pedidos.aggregate([  
  { $match: { STATUS: "Shipped" } },  
  { $group: { _id: "$COUNTRY", receita: { $sum: "$SALES" } } },  
  { $sort: { receita: -1 } },  
  { $limit: 3 }  
])
```

Exercício para o aluno: Construa um pipeline que devolva a linha de produto (PRODUCTLINE) com a maior receita total, considerando apenas pedidos de 2004.

Exercício 28 — \$addToSet e contagem de únicos

O que faz: \$addToSet acumula valores únicos num array durante o \$group. Combinado com \$size, permite contar distintos.

Exemplo — quantos clientes distintos por país:

```
db.pedidos.aggregate([
  { $group: { _id: "$COUNTRY", clientes: { $addToSet: "$CUSTOMER_ID" } } },
  { $project: { _id: 1, nClientes: { $size: "$clientes" } } }
])
```

Exercício para o aluno: Para cada PRODUCTLINE, conte quantos países distintos compraram esse produto.

Exercício 29 — Índice composto

O que faz: Um índice em vários campos otimiza queries que filtram/ordenam por essa combinação. A ordem dos campos importa.

Exemplo:

```
db.pedidos.createIndex({ COUNTRY: 1, ORDERDATE: -1 })
```

Exercício para o aluno: Crie um índice composto por STATUS (crescente) e SALES (decrecente) e execute uma query que o utilize. Confirme com .explain("executionStats") que o índice foi usado.

Exercício 30 — insertOne e insertMany

O que faz: Inserem um ou vários documentos. Se _id não for fornecido, o MongoDB gera automaticamente.

Exemplo:

```
db.pedidos.insertOne({
  ORDERNUMBER: 10200, QUANTITYORDERED: 40, PRICEEACH: 95,
  SALES: 3800, STATUS: "Shipped", ORDERDATE: "2005-11-01",
  PRODUCTLINE: "Motorcycles", PRODUCTCODE: "S10_1678",
  CUSTOMER_ID: 1, CONTACTLASTNAME: "Nelson",
  CITY: "Nantes", COUNTRY: "France"
})
```

Exercício para o aluno: Insira **três** novos pedidos com ORDERNUMBER 10201, 10202 e 10203 usando insertMany. Depois, execute countDocuments() para confirmar que ficaram 38 documentos.

Bónus — Boas práticas ao resolver

- Antes de qualquer updateMany/deleteMany, execute o mesmo filtro com find() para verificar o que será afetado.
- Use .pretty() no shell para saída mais legível.
- Para pipelines longos, guarde o array numa variável: const p = [{...}, {...}]; db.pedidos.aggregate(p).
- Utilize .explain("executionStats") para perceber o plano de execução e o uso dos índices.